📄 Task Manager Codebase Understanding – Summary Document

Initial vs Final Understanding

Initial Understanding:
At first glance, I identified the project as a simple task management application built using Python. I observed a modular folder structure with separate files for logic, models, and storage. I assumed it used basic Python libraries and possibly JSON for data storage. However, I was not fully clear on how components interacted or where specific features were implemented.

Final Understanding:
After deeper exploration and using AI prompts, I now understand that the application follows a clean, modular architecture. The TaskManager acts as the central controller handling business logic, while Task represents the domain model. The TaskStorage component abstracts data persistence, likely using a JSON file. The CLI (cli.py) serves as the entry point for user interaction. The system uses enums (TaskStatus, TaskPriority) to enforce valid states, and overall follows good separation of concerns.

Key Insights from Each Prompt

1. Project Structure Prompt:

- Helped confirm the architecture is modular and well-organized
- Identified cli.py as the entry point
- Highlighted clear separation between logic, storage, and models
- Corrected my assumption that there might be frameworks involved (it's pure Python)

2. Feature Location Prompt:

- Showed that new features (like exporting tasks) should be implemented in TaskManager and possibly TaskStorage
- Helped identify reusable patterns like how tasks are retrieved and processed
- Clarified that file-related logic belongs in the storage layer

3. Domain Understanding Prompt:

- Helped me understand relationships between entities (Task, TaskStatus, TaskPriority)
- Reinforced that business logic is split between the model (e.g., mark_as_done) and manager

- Improved my understanding of how task data flows through the system

---

Approach to Implementing New Business Rule

Business Rule:
Tasks overdue for more than 7 days should be automatically marked as *abandoned*, unless they are high priority.

Files to Modify:

- task_manager.py → for applying the rule logic
- models.py → possibly to add a new status (ABANDONED)
- storage.py → ensure updated tasks are saved

Implementation Plan:

1. Add a new status (ABANDONED) in TaskStatus
2. Create a method to check overdue duration (using datetime)
3. Loop through tasks (likely in get_statistics() or a new method)
4. If a task is overdue > 7 days and not high priority:
    - Update status to ABANDONED
5. Save changes using TaskStorage.save()

Questions for Team:

- Should this rule run automatically or only when triggered?
- Is "abandoned" a permanent state or reversible?
- Should users be notified when a task is abandoned?

Reflection

How AI Prompts Helped:
AI prompts helped structure my thinking and guided me to focus on architecture, data flow, and feature location. They made it easier to break down unfamiliar code into understandable parts and validate my assumptions quickly.

Remaining Uncertainties:

- Exact implementation details inside storage.py

- How tasks are serialized/deserialized
- Whether there are hidden edge cases in status transitions

Next Steps:

- Review storage.py in detail
- Run the application and observe behavior
- Write small test cases to experiment with features

Personal Learning Strategy

From this exercise, I've developed a clearer approach to understanding new codebases:

- Start with structure before diving into code
- Identify entry points and core components
- Trace data flow (input → logic → storage)
- Use AI to validate and refine understanding
- Experiment with small changes to confirm assumptions

Final Thought

This exercise helped me move from surface-level understanding to a deeper, more practical grasp of the system. I now feel more confident navigating and contributing to unfamiliar codebases.